



CORSO DI STUDIO IN
Ingegneria Informatica

Report Progetto SOA

Simone Ercole
Matricola: 0360105

Indice

1	Specifica del Progetto	2
2	Requisiti Tecnici e Ambiente di Esecuzione	2
3	Panoramica dell'Architettura	3
3.1	Componenti in Kernel-Space	3
3.2	Applicativi in User-Space	4
4	Sistema di Compilazione e Gestione del Ciclo di Vita (Makefile)	4
5	Il Motore di Intercettazione: <code>hook.c</code>	5
5.1	Risoluzione dinamica dei simboli del kernel	5
5.2	Tecnica di binary patching	5
5.3	Architettura del Trampolino di Intercettazione (<code>hook_func</code>)	6
5.4	Sicurezza in ambiente multiprocessore	7
5.5	Smontaggio Sicuro	7
5.5.1	Automazione dello scaricamento (<code>safe_rmmod.sh</code>)	8
6	Motore di Rate-Limiting: <code>throttle.c</code>	8
6.1	Filtraggio e Implementazione del Token Bucket	8
6.2	Accodamento e Prevenzione del Thundering Herd	9
6.3	Gestione delle Statistiche	10
7	L'Interfaccia di Controllo e Comunicazione: <code>ioctl1.c</code>	11
7.1	Modello di Autorizzazione	11
7.2	Ottimizzazione Concorrente e Lockless Fast-Path	11
7.3	Sanificazione degli Input	12
8	Il Device Driver: <code>dev.c</code>	12
8.1	Creazione del Nodo e Delega dei Permessi	12
8.2	Gestione degli Errori di Inizializzazione e Teardown	13
9	Ciclo di Vita del Modulo: <code>module.c</code>	13
9.1	Sequenza di Inizializzazione	13
9.2	Sequenza di Terminazione	13
10	Applicativi in User-Space: Configurazione e Validazione	14
10.1	Interfaccia di Amministrazione (<code>client.c</code>)	14
10.2	Suite di Test e Validazione (<code>test_throttle.c</code>)	14

1 Specifica del Progetto

Il progetto prevede lo sviluppo di un Linux Kernel Module per la limitazione della frequenza di esecuzione delle chiamate di sistema.

Il modulo espone un *device driver* in grado di supportare la registrazione e la rimozione dinamica di:

- nomi di programmi;
- user ID;
- numeri di system call, secondo lo standard x86-64.

L'interazione con il device driver avviene tramite la system call `ioctl()`. Quando una delle system call registrate viene invocata da un programma o da un utente monitorato, l'esecuzione della chiamata è soggetta a supervisione da parte del modulo. In particolare, il sistema consente di configurare un limite massimo di invocazioni consentite in una finestra temporale di un secondo. Se tale volume viene superato, i thread invocanti sono temporaneamente sospesi, ritardando l'esecuzione effettiva della system call. Il modulo supporta sia syscall di natura bloccante sia non bloccante.

Il driver soddisfa inoltre i seguenti requisiti operativi e di sicurezza:

- **Permessi.** L'aggiornamento dei parametri di configurazione è consentito esclusivamente a thread in esecuzione con privilegi di amministratore, mentre l'interrogazione dello stato corrente è accessibile in sola lettura a qualsiasi utente.
- **Stato del monitor.** È possibile attivare o disattivare il monitor in qualsiasi istante. In stato inattivo non viene applicato alcun limite.
- **Statistiche.** Il sistema calcola e restituisce il ritardo di picco accumulato (con il relativo UID e nome del programma) e il numero medio e massimo di thread bloccati.

2 Requisiti Tecnici e Ambiente di Esecuzione

Per garantire la corretta compilazione e il funzionamento del modulo, l'ambiente di esecuzione deve soddisfare i seguenti requisiti:

- **Architettura Hardware:** Processore x86-64 con supporto ai sistemi multiprocessore.
- **Kernel Linux:** Versione minima 6.9.

- **Configurazioni Kernel (Kconfig):** Il kernel host deve essere stato compilato con le seguenti opzioni abilitate:
 - `CONFIG_MODULES=y`: Supporto per il caricamento di moduli.
 - `CONFIG_KPROBES=y` e `CONFIG_KALLSYMS=y`: Necessari per la risoluzione dinamica degli indirizzi non esportati.
 - `CONFIG_SMP=y`: Supporto alle primitive *per-CPU* e ai lock multiprocessore.
 - `CONFIG_HIGH_RES_TIMERS=y`: Infrastruttura `hrtimer` per il *rate-limiting*.
- **Sicurezza di Sistema (Secure Boot):** È necessario che la funzionalità UEFI Secure Boot sia disabilitata. In alternativa, il binario compilato deve essere preventivamente firmato utilizzando una chiave crittografica registrata nel database MOK del sistema.
- **Intestazioni di Sistema:** Pacchetto `linux-headers` installato e allineato all'esatta versione del kernel in esecuzione.
- **Toolchain C:** GNU Make e GCC per la compilazione standard della base di codice.
- **AWK:** Utilizzato dal `Makefile` in fase di compilazione per estrarre dinamicamente la mappatura degli identificatori delle system call dall'header di sistema `unistd_64.h`.

3 Panoramica dell'Architettura

Il sistema è architettato su due livelli disaccoppiati: il livello kernel, che incapsula la logica di intercettazione e regolazione, e il livello utente, destinato all'interazione e al testing.

3.1 Componenti in Kernel-Space

- **Ciclo di Vita (`module.c`):** Orchestra le routine di inizializzazione e smantellamento del modulo.
- **Gestione Device (`dev.c`):** Registra il *character device* ed espone il nodo fisico necessario per le comunicazioni provenienti dallo spazio utente.
- **Interfaccia di Configurazione (`ioctl.c`):** Riceve i comandi dallo spazio utente, valida i privilegi di sicurezza e aggiorna le tabelle di filtraggio.
- **Motore di Intercettazione (`hook.c`):** Manipola dinamicamente la memoria eseguibile del kernel per deviare il flusso delle system call monitorate verso l'infrastruttura di controllo.

- **Motore di Rate-Limiting (`throttle.c`):** Decide le sorti dei thread intercettati applicando un algoritmo *Token Bucket lockless*. Gestisce le code di attesa, i timer di risveglio e la raccolta isolata delle statistiche *per-CPU*.

3.2 Applicativi in User-Space

- **Client Amministrativo (`client.c`):** Interfaccia a riga di comando che incapsula le chiamate `ioctl`, consentendo di configurare le soglie operative, aggiornare l'elenco dei target e consultare la telemetria.
- **Suite di Validazione (`test_throttle.c`):** Programma multi-thread concepito per collaudare il modulo kernel.

4 Sistema di Compilazione e Gestione del Ciclo di Vita (Makefile)

Il processo di build del progetto è centralizzato e gestito tramite un **Makefile** strutturato, il quale si occupa di orchestrare sia la compilazione del modulo kernel sia la creazione degli applicativi in spazio utente.

Una particolarità del processo di compilazione è rappresentata dal target **`generate_headers`**. Prima di avviare la compilazione del codice sorgente in C, il Makefile esegue automaticamente lo script bash **`generate_mapping.sh`**. Tale script ispeziona le directory di sistema alla ricerca dell'header **`unistd_64.h`** e, tramite l'utilizzo del comando **`awk`**, estrae dinamicamente gli identificatori numerici e i nomi testuali di tutte le system call supportate dall'architettura ospite.

Questi dati vengono formattati e scritti all'interno del file autogenerato **`header/syscall_mapping.h`**, popolando un array dimensionato dinamicamente in base all'ID massimo individuato. Questo approccio garantisce che il modulo sia sempre allineato all'esatta mappatura delle system call del kernel sul quale viene compilato.

Oltre ai target **`client`** e **`test`** per gli applicativi user-space e al comando di pulizia **`clean`**, il Makefile definisce due target di servizio per semplificare l'amministrazione:

- **`make ins`:** carica il modulo compilato nel kernel tramite **`insmod`**.
- **`make rem`:** esegue lo script **`safe_rmmod.sh`** per rimuovere in sicurezza il modulo, come illustrato nella Sezione 5.5.1.

5 Il Motore di Intercettazione: `hook.c`

Questa sezione descrive il componente del modulo responsabile dell'intercettazione delle system call mediante alterazione dinamica del codice in memoria, garantendo totale sicurezza in un ambiente multiprocessore.

5.1 Risoluzione dinamica dei simboli del kernel

Il modulo richiede l'accesso a strutture e funzioni interne del kernel non esportate ai moduli esterni, tra cui `sys_call_table`, `x64_sys_call`, `kallsyms_lookup_size_offset` e le API di patching della memoria (`text_poke_bp` o la sua variante moderna `smp_text_poke_single`). A partire dal kernel 5.7, anche il simbolo `kallsyms_lookup_name`, che permetterebbe di risolvere direttamente tali indirizzi, non è più esportato.

Per ottenerne comunque l'indirizzo, il modulo registra temporaneamente una `kprobe` sul nome `kallsyms_lookup_name`: il sottosistema delle `kprobe` dispone di accesso alla tabella dei simboli del kernel indipendentemente dalle regole di esportazione, e deve pertanto risolvere l'indirizzo del simbolo per poter agganciare la sonda. Una volta ottenuto l'indirizzo dalla struttura `kprobe`, la sonda viene deregistrata e il puntatore a `kallsyms_lookup_name` viene impiegato per risolvere dinamicamente tutti gli altri simboli necessari al funzionamento del modulo.

5.2 Tecnica di binary patching

Dal kernel 6.9, il dispacciamento delle system call non avviene più tramite lettura indiretta di `sys_call_table`, ma è gestito da una funzione generata dal compilatore, `x64_sys_call`, la cui struttura a `switch` produce un'istruzione di salto (`CALL` o `JMP`) distinta per ciascuna syscall. È questa proprietà a rendere possibile la tecnica di *inline patching* adottata dal modulo. La procedura di inserimento dell'hook si articola nei seguenti passaggi:

1. **Recupero dell'indirizzo originale.** L'indirizzo della funzione di sistema originale viene letto direttamente da `sys_call_table[syscall_id]`.
2. **Scansione della memoria eseguibile.** La dimensione esatta di `x64_sys_call` viene calcolata dinamicamente tramite `kallsyms_lookup_size_offset`. Il corpo della funzione viene poi analizzato byte per byte, alla ricerca degli opcode `0xE8` (`CALL`) o `0xE9` (`JMP`).
3. **Decodifica dell'offset.** Per ogni candidato individuato, i 4 byte successivi vengono interpretati come intero con segno a 32 bit; sommando tale offset all'indirizzo dell'istruzione successiva si ottiene la destinazione effettiva del salto.
4. **Verifica della corrispondenza.** Si confronta la destinazione calcolata con l'indirizzo originale ottenuto al punto 1: in caso di coincidenza, si è individuato il punto di aggancio corretto.

5. **Calcolo del nuovo offset.** Si calcola la distanza relativa tra l'istruzione individuata e l'indirizzo della funzione trampolino `hook_func`.
6. **Composizione e Sovrascrittura.** Poiché l'API di patching del kernel necessita di decodificare l'istruzione per emularla in modo sicuro, viene predisposto un buffer di 5 byte contenente l'opcode originale e il nuovo offset. L'intera istruzione di 5 byte viene quindi sostituita delegando l'operazione alla funzione di patching risolta dinamicamente in fase di inizializzazione (`text_poke_bp` o `smp_text_poke_single`).

Prima della modifica, i 5 byte originali dell'istruzione e l'indirizzo di memoria interessato sono salvati nella struttura `patch_sites`, per consentirne il ripristino in fase di rimozione dell'hook.

5.3 Architettura del Trampolino di Intercettazione (`hook_func`)

A seguito dell'iniezione del trampolino, qualsiasi invocazione delle system call bersaglio devia il flusso di esecuzione verso la funzione wrapper `hook_func`. La routine gestisce l'intercettazione attraverso quattro fasi sequenziali, strutturate per garantire trasparenza operativa e integrità del kernel:

1. **Estrazione del contesto:** La funzione esamina la struttura `pt_regs` passata dall'architettura di trap a basso livello ed estrae dal registro hardware (`orig_ax` su x86_64) il numero identificativo della syscall intercettata.
2. **Acquisizione del riferimento:** Prima di eseguire qualsiasi logica interna, il thread invoca la primitiva nativa `try_module_get(THIS_MODULE)`, incrementando atomicamente il contatore dei riferimenti. Se l'invocazione fallisce, significa che il kernel ha già avviato la fase di scaricamento del modulo. In questo scenario, l'hook adotta una strategia di *transparent retry* restituendo il codice speciale `-ERESTARTSYS`. Questa scelta istruisce il sottosistema del kernel a decrementare il *Program Counter* del thread prima del rientro in *user space*: l'applicazione rieseguirà così la system call pochi cicli di clock dopo, trovando l'hook rimosso e transitando in modo trasparente verso l'handler nativo originale.
3. **Filtraggio, Throttling e Invocazione nativa:** Ottenuta la certezza di persistenza del modulo, il flusso chiama il motore di rate-limiting tramite `throttle_check()`. Qualora l'invocazione richieda un contenimento, il task viene sospeso nella *wait queue* fino al ripristino della quota consentita. Al termine della fase di controllo, il wrapper richiama l'handler originale prelevato dalla `sys_call_table`.
4. **Rilascio controllato del riferimento:** Solo a valle della conclusione della system call originale (e in qualsiasi ramo di uscita anticipata), il thread esegue `module_put(THIS_MODULE)`. Questa sequenza garantisce che il *refcount* rimanga maggiore di zero per l'intera durata della chiamata (inclusi eventuali blocchi pro-

lungati su operazioni di I/O), rendendo impossibile la deallocazione del segmento di codice durante l'esecuzione del task.

5.4 Sicurezza in ambiente multiprocessore

L'alterazione di codice eseguibile a runtime in un'architettura multiprocessore espone al rischio critico che un'altra CPU esegua o prelevi in pipeline l'istruzione esatta nel momento in cui questa viene sovrascritta, generando comportamenti indefiniti o *Kernel Panic*.

Per garantire l'atomicità e la totale sicurezza dell'operazione, il modulo si avvale dell'API ufficiale del kernel per il patching "a caldo". Poiché le interfacce interne del kernel sono soggette a continui aggiornamenti, il modulo implementa un meccanismo di adattamento dinamico: tenta inizialmente di utilizzare l'API `text_poke_bp()` e, qualora non fosse disponibile, ripiega sulla sua evoluzione `smp_text_poke_single()`.

Entrambe le funzioni orchestrano il medesimo meccanismo di sincronizzazione basato sulle interruzioni software. Il processo inizia sostituendo innanzitutto il primo byte dell'istruzione bersaglio con un breakpoint. Fatto ciò, viene inviato un *Inter-Processor Interrupt* per forzare lo svuotamento delle code di prefetch su tutti gli altri core. A questo punto, la funzione procede in modo sicuro alla scrittura dei restanti byte dell'offset e conclude ripristinando l'opcode originale al posto del breakpoint, operazione seguita da un'ulteriore sincronizzazione globale.

Qualora una CPU concorrente tentasse di eseguire l'istruzione proprio durante questa finestra di transizione, l'attivazione del breakpoint ne sospenderebbe temporaneamente il flusso. L'handler interno del kernel preposto alla gestione dell'interruzione si farebbe quindi carico di emulare l'istruzione originale, sfruttando le informazioni contenute nel buffer di 5 byte fornito in pasto all'API dal modulo, per poi far riprendere l'esecuzione al core interessato in totale trasparenza.

5.5 Smontaggio Sicuro

Rimuovere un modulo che intercetta chiamate di sistema richiede cautela: deallocare il codice mentre un thread sta eseguendo l'hook — o è sospeso in attesa di I/O nella funzione originale — provocherebbe un *Kernel Panic* al rientro dell'esecuzione su un'area di memoria liberata.

La sicurezza dello scaricamento è affidata al meccanismo di *reference counting* nativo del kernel Linux.

Grazie all'uso congiunto di `try_module_get()` e `module_put()`, il kernel tiene traccia in modo atomico di ogni thread che attraversa l'hook, per qualunque system call registrata. All'invocazione di `rmmmod`, la syscall `delete_module` controlla il contatore dei riferimenti: se questo è maggiore di zero l'operazione viene respinta subito con errore `-EBUSY`, impedendo l'esecuzione della funzione di rimozione del modulo.

5.5.1 Automazione dello scaricamento (`safe_rmmod.sh`)

Per evitare rimozioni manuali e gestire in modo pulito gli scenari ad alto traffico, è stato creato lo script `safe_rmmod.sh`, integrato direttamente nel target `make rem`.

Lo script automatizza la procedura di scaricamento coordinando tre fasi successive:

1. **Ripristino degli hook e svuotamento delle code:** Utilizzando il programma `client`, lo script recupera la lista delle syscall registrate e ne richiede la rimozione, ripristinando le istruzioni originali del dispatcher nativo. Contestualmente disattiva il monitoraggio, operazione che invia una notifica di risveglio a tutti i thread sospesi nella *wait queue*. In questo modo viene bloccato l'ingresso a nuove chiamate e i thread già accodati possono completare l'esecuzione, permettendo al contatore dei riferimenti di scendere verso lo zero.
2. **Tentativo di rimozione del modulo:** Completata la prima fase, lo script prova a scaricare il modulo invocando `rmmod`. Se il contatore dei riferimenti è effettivamente sceso a zero, il kernel autorizza l'operazione ed esegue la funzione di uscita `my_module_exit`. Se invece il comando viene respinto, il flusso prosegue con la successiva fase di diagnostica.
3. **Diagnostica e gestione dei task bloccati:** Se `rmmod` fallisce comunque con errore `-EBUSY`, significa che uno o più task, pur avendo superato il controllo di throttling, sono correntemente sospesi all'interno dell'handler originale del kernel, trattenendo il riferimento al modulo. Lo script interroga il file `refcnt` in *sysfs* e analizza gli stack di esecuzione in `/proc/*/task/*/stack` per individuare i PID coinvolti; mostra quindi le informazioni all'operatore e consente, previa conferma esplicita, di inviare un segnale `SIGTERM` per sbloccare i processi e finalizzare la rimozione.

6 Motore di Rate-Limiting: `throttle.c`

Il componente `throttle.c` implementa il nucleo logico del rate-limiting e la raccolta della telemetria. La funzione principale, `throttle_check()`, viene invocata per ogni system call monitorata e determina se consentire l'esecuzione immediata o forzare la sospensione del thread.

6.1 Filtraggio e Implementazione del Token Bucket

L'esecuzione di `throttle_check()` si apre con un filtraggio preliminare. Verificata l'attivazione globale del monitor, il thread estrae le proprie credenziali e interroga le tabelle hash di registrazione. Se l'eseguibile o l'utente non sono soggetti a restrizioni, il processo viene scartato dal controllo ed esce immediatamente senza subire latenze.

I processi monitorati, invece, interagiscono con un *Token Bucket* implementato senza usare lock bloccanti, in cui i token residui sono mantenuti nella variabile atomica `g_available_tokens`.

La fase di **ricarica** scatta al variare del secondo temporale. Il thread campiona il tempo di sistema tramite `ktime_get_seconds()` e lo confronta con il timestamp dell'ultimo aggiornamento (`g_last_second`). Se i valori differiscono, un nuovo secondo è iniziato. Per gestire la concorrenza di più thread che rilevano simultaneamente questo scatto temporale, l'aggiornamento è mediato dalla primitiva `atomic64_cmpxchg`: solo il processo che riesce a sovrascrivere per primo `g_last_second` ottiene il diritto esclusivo di ripristinare i token e risvegliare la *wait queue*.

Per massimizzare la reattività, questa routine di ricarica è innescata da due meccanismi complementari:

- **Ricarica Sincrona (Opportunistica):** Prima di acquisire un gettone, il thread stesso invoca proattivamente la logica di ripristino (`maybe_refill_tokens()`). Ciò permette al sistema di reagire al cambio temporale sfruttando direttamente il traffico delle system call, senza attendere l'interrupt del timer.
- **Ricarica Asincrona (Timer):** A supporto dei thread agisce un *High-Resolution Timer*. Per evitare overhead a sistema scarico, il timer è armato *on-demand* solo quando i token si esauriscono e un thread sta per essere bloccato. Una volta attivo, il timer continua a riavviarsi finché il monitor resta acceso, garantendo lo sblocco della *wait queue* anche in assenza di nuovo traffico.

Terminata l'eventuale ricarica, si passa al **consumo**: il thread tenta di sottrarre un token tramite `atomic_add_unless(&g_available_tokens, -1, 0)`, che decrementa atomicamente il contatore solo se strettamente positivo. Se l'operazione ha successo, il limite non è stato raggiunto e l'esecuzione nativa prosegue. In caso contrario, il thread si predispone alla sospensione.

6.2 Accodamento e Prevenzione del Thundering Herd

Quando un thread trova il secchio dei token vuoto, registra il timestamp di ingresso, incrementa il contatore globale `g_blocked_threads_count` e si sospende.

L'attesa è gestita tramite la primitiva `wait_event_interruptible_exclusive()`. L'uso della modalità *exclusive* (flag `WQ_FLAG_EXCLUSIVE`) serve a evitare il fenomeno del *thundering herd*. Al momento del ripristino, la funzione di sblocco invoca `__wake_up()` passando `current_max` (la quota massima di syscall ammesse ogni secondo) come numero massimo di task esclusivi da risvegliare. Dato che tutti i thread in coda sono registrati come esclusivi, il kernel risveglia solo il numero esatto di processi che possono effettivamente ottenere un token, azzerando i cambi di contesto inutili causati da risvegli a vuoto.

6.3 Gestione delle Statistiche

Le metriche di monitoraggio vengono aggiornate dai thread in esecuzione in due momenti del loro ciclo di vita: all'inizio della fase di accodamento (per tracciare il numero di processi bloccati) e al termine della stessa (per registrare la latenza subita).

Poiché queste scritture avvengono ad alta frequenza, l'adozione di variabili globali condivise degraderebbe le prestazioni complessive del sistema a causa della continua invalidazione delle linee di cache e della contesa sui lock. Per preservare la scalabilità su architetture multiprocessore, l'architettura alloca le metriche localmente su ciascuna CPU, mediante la struttura dati `throttle_stats_percpu`. I dati registrati nei vari core vengono poi aggregati in un'unica vista globale su richiesta di lettura da parte dello *user space*.

Al fine di garantire l'integrità totale dei record ed evitare fenomeni di letture intermedie corrotte durante l'aggregazione, l'accesso ai dati è regolato tramite il costrutto `u64_stats_sync`. Questo pattern di sincronizzazione, ottimizzato per il kernel Linux, si basa su un *sequence counter* numerico e si articola sui seguenti principi:

- **Percorso di Scrittura:** L'aggiornamento delle metriche locali all'inizio e alla fine del blocco è incapsulato tra le primitive `u64_stats_update_begin` e `_end`. La prima macro incrementa il contatore di sequenza rendendolo dispari, segnalando l'inizio di una transazione; la seconda lo incrementa nuovamente riportandolo a un valore pari, a indicare la fine della modifica. L'intero blocco è protetto da `local_irq_save()`, che disabilita gli interrupt locali per impedire la migrazione del thread o l'intervento di handler asincroni, garantendo un accesso esclusivo per-CPU.
- **Percorso di Lettura:** Quando lo *user space* richiede le statistiche, la fase di aggregazione interroga le strutture di tutte le CPU sfruttando cicli `do/while`. La macro `u64_stats_fetch_begin` campiona il contatore prima dell'estrazione dei dati, mentre `_retry` lo riverifica al termine. Se il valore campionato inizialmente è dispari (scrittura in corso) o se differisce dal contatore finale, i dati prelevati vengono considerati "sporchi" e rigettati: il ciclo viene ripetuto localmente finché la lettura non risulta coerente, il tutto senza mai acquisire lock bloccanti.
- **Sincronizzazione via IPI:** L'integrità del modello per-CPU impone che la memoria di ciascuna istanza sia modificata esclusivamente dal core proprietario. Una richiesta di azzeramento delle statistiche avanzata dallo *user space* non viene quindi applicata iterativamente da un'unica CPU, ma delegata all'infrastruttura `on_each_cpu()`. Questa primitiva invia un *Inter-Processor Interrupt* a tutti i processori attivi, forzando ciascun core a eseguire la routine di azzeramento localmente.

7 L'Interfaccia di Controllo e Comunicazione: `ioctl.c`

Il componente `ioctl.c` realizza l'interfaccia di comunicazione tra lo spazio utente e il kernel. Ricevendo le direttive attraverso la system call `ioctl()`, si occupa della validazione dei privilegi, della sanificazione degli input e della gestione concorrente delle strutture dati necessarie al tracciamento dei target monitorati.

7.1 Modello di Autorizzazione

L'architettura applica una distinzione sui privilegi: le operazioni di sola lettura sono concesse a qualsiasi utente, le operazioni che alterano lo stato del modulo sono invece riservate esclusivamente agli utenti privilegiati.

La funzione di dispacciamento valuta la natura del comando e, per le operazioni di scrittura, impone che l'identificatore utente effettivo corrisponda all'amministratore. Questo controllo non si limita a un semplice confronto numerico diretto, ma si avvale della primitiva `uid_eq()` confrontata con la costante `GLOBAL_ROOT_UID`. Questa scelta è fondamentale nei sistemi Linux basati su container: un processo confinato all'interno di uno *user namespace* isolato può presentarsi localmente con UID pari a zero, senza per questo disporre dei privilegi di amministratore sul sistema host. Verificando l'identità rispetto allo *user namespace* iniziale, il modulo garantisce che solo l'amministratore effettivo del sistema possa modificarne la configurazione, respingendo tentativi di alterazione da parte di processi confinati.

7.2 Ottimizzazione Concorrente e Lockless Fast-Path

Le liste dei target da monitorare sono organizzate tramite tabelle hash del kernel. L'architettura prevede una separazione tra le operazioni di interrogazione e quelle di aggiornamento, con l'obiettivo di abbattere la *lock contention*.

- **Dominio di lettura (Fast Path Lockless):** Il modulo è intrinsecamente *read-intensive*, poiché la verifica dei target avviene ogni volta che il motore di rate-limiting viene interrogato. Per evitare di degradare le prestazioni complessive del sistema operativo, le letture sono gestite interamente tramite le primitive RCU. L'architettura RCU è ideale per questo scenario: permette a molteplici thread di leggere i dati in modo rapido e simultaneo, senza mai dover acquisire un lock bloccante.
- **Dominio di scrittura (Mutex):** Le operazioni di modifica sono meno frequenti rispetto alle letture, ma devono essere esclusive per evitare corruzioni delle strutture dati, motivo per cui richiedono l'acquisizione di un `mutex`. Poiché l'allocazione dinamica di memoria necessaria per i nuovi target può causare la sospensione del thread chiamante, l'allocazione viene eseguita preventivamente *prima* di acquisire il

lock. In questo modo, la sezione critica viene occupata esclusivamente per i pochi cicli di clock necessari ad aggiornare i puntatori della lista, minimizzando la latenza per gli altri processi in attesa.

Durante le operazioni di deregistrazione, la rimozione della memoria pregressa utilizza il costrutto `kfree_rcu()`: il nodo rimosso viene immediatamente sganciato dalla lista logica, ma la deallocazione fisica della memoria è posticipata in background al termine di eventuali letture residue ancora in corso su altri processori.

7.3 Sanificazione degli Input

Tutte le interazioni provenienti dallo spazio utente vengono validate per preservare l'integrità del kernel. Il trasferimento dei parametri è delegato alle primitive `copy_from_user` e `copy_to_user`, progettate per gestire eventuali *page fault* derivanti da puntatori invalidi.

Oltre alla validazione della memoria, il modulo esegue un controllo semantico sui domini dei dati ricevuti:

- **Prevenzione Out-of-Bounds:** L'ID della syscall passato come argomento viene verificato contro i limiti strutturali dell'array `syscall_sym_names` e scartato qualora risulti negativo o superiore al massimo consentito.
- **Sicurezza delle Stringhe:** Un'attenzione è dedicata anche all'acquisizione dei nomi degli eseguibili. Il modulo rifiuta i payload contenenti stringhe vuote o sprovviste del terminatore nullo entro il limite massimo di `MAX_PROG_NAME`.
- **Coerenza Logica:** I parametri destinati a configurare gli User ID o le soglie massime di limitazione vengono invalidati se assumono valori negativi, garantendo il mantenimento di uno stato interno coerente.

8 Il Device Driver: `dev.c`

Il componente `dev.c` si occupa di istanziare e registrare il *character device* nel kernel, esponendo il nodo `/dev/throttle_dev` che lo spazio utente utilizza per comunicare con il modulo. L'interfaccia accetta esclusivamente comandi tramite `ioctl`.

8.1 Creazione del Nodo e Delega dei Permessi

Gli identificatori del device (major e minor number) sono allocati dinamicamente, per evitare conflitti con altri device presenti nel sistema. La gestione dei permessi di accesso richiede invece un intervento esplicito.

Per impostazione predefinita, la funzione del kernel che crea il nodo impone permessi restrittivi. Questo impedirebbe l'apertura del device da parte di utenti non privilegiati,

anche per le sole operazioni di lettura consentite dalla specifica. Per garantire l'accessibilità richiesta, il modulo installa una funzione di callback che imposta i permessi del nodo a 0666.

Questa apertura sposta il controllo degli accessi dal VFS al livello applicativo: l'apertura del file è consentita a chiunque, mentre spetta a `ioctl.c` determinare, per ciascun comando, se il chiamante dispone dei privilegi necessari.

8.2 Gestione degli Errori di Inizializzazione e Teardown

L'inizializzazione del device si articola in più fasi: allocazione dei numeri identificativi, registrazione del `cdev`, creazione della classe in `sysfs` e creazione del nodo. Se una di queste fasi fallisce, le risorse allocate nelle fasi precedenti devono essere rilasciate esplicitamente.

A questo scopo, la funzione di setup implementa un rollback a cascata: in caso di errore in una fase, il codice utilizza salti condizionati per disfare esclusivamente le allocazioni completate con successo nelle fasi precedenti. Analogamente, la rimozione del modulo tramite `rmmmod` esegue il teardown delle risorse nell'ordine esattamente inverso rispetto alla creazione, garantendo che il kernel non acceda mai a componenti parzialmente smantellati.

9 Ciclo di Vita del Modulo: `module.c`

Il componente `module.c` coordina il ciclo di vita tramite le routine `module_init` e `module_exit`.

9.1 Sequenza di Inizializzazione

La procedura di avvio segue un ordine di dipendenza preciso. Inizializza dapprima le strutture temporali del rate-limiting, quindi risolve i simboli del kernel e inizializza il motore di intercettazione.

Solo quando l'infrastruttura di intercettazione è pienamente operativa, il modulo espone il *character device*. Questo ordine impedisce allo spazio utente di inviare comandi `ioctl` mentre il sottosistema di intercettazione non è ancora pronto. Il fallimento di una qualsiasi fase intermedia interrompe l'avvio, restituendo immediatamente il codice di errore a `insmod`.

9.2 Sequenza di Terminazione

La routine di rimozione smantella il modulo in cinque fasi sequenziali:

1. **Chiusura dell'accesso dallo user-space:** L'invocazione di `cleanup_monitor_device()` distrugge il nodo `/dev/throttle_dev`, impedendo la ricezione di nuovi comandi `ioctl` durante la fase di spegnimento.

2. **Rimozione degli hook:** Il modulo disinstalla tutte le patch binarie iniettate in memoria, ripristinando il normale flusso d'esecuzione per le system call interessate e chiudendo fisicamente l'ingresso a nuove intercettazioni.
3. **Disattivazione e sblocco:** Tramite l'invocazione di `throttle_set_monitor(false)`, il sistema si assicura che il monitor venga spento.
4. **Arresto del timer:** Attraverso `throttle_cleanup_timer()`, la cancellazione sincrona del timer ad alta risoluzione avviene in sicurezza, senza più alcun rischio di riarmo.
5. **Deallocazione della memoria logica:** L'ultima fase, `cleanup_hashtables()`, svuota le tabelle hash deallocando i record allocati per system call, programmi e UID.

10 Applicativi in User-Space: Configurazione e Validazione

L'interazione con il modulo kernel è demandata a due applicativi user-space: un client per la gestione amministrativa e una suite di test multi-thread per la validazione sotto stress.

10.1 Interfaccia di Amministrazione (`client.c`)

Il programma `client.c` implementa un'interfaccia a riga di comando per la configurazione e il monitoraggio del modulo in spazio utente.

L'interazione con il kernel avviene tramite l'apertura del *character device* `/dev/throttle_dev` e l'utilizzo della system call `ioctl`. Il ruolo del client è strettamente limitato a fare da intermediario: traduce i comandi testuali e gli argomenti forniti dall'utente nei codici operativi e nelle strutture dati attese dal modulo.

Infine, il programma si occupa di stampare in formato leggibile i dati estratti dal kernel e di fornire un riscontro immediato sull'esito delle operazioni, traducendo i codici di errore del kernel in messaggi diagnostici tramite la funzione `perror()`.

10.2 Suite di Test e Validazione (`test_throttle.c`)

Per validare l'architettura del modulo è stata sviluppata una suite di test in grado di generare condizioni di stress mirato. L'applicativo comprende le seguenti modalità esecutive:

- **Carico, impatto architetturale e calibrazione dinamica (`performancetest`).**
Questo test è progettato per misurare l'impatto prestazionale introdotto dalla logica

di *hooking* sull'architettura del sistema. Generando un traffico intensivo, il test calcola parametri come il tempo di esecuzione (medio, minimo e massimo) per singola system call e il *rate* globale. Sfruttando una prima fase di *baseline* a sistema pulito, l'applicativo calcola una soglia di latenza dinamica autocalibrata sull'hardware ospite. Questa soglia viene poi utilizzata nella seconda fase (con throttling attivo) per monitorare quante chiamate sono state effettivamente bloccate e accodate, isolando l'overhead dell'hook dal ritardo imposto dal limitatore.

- **Stress delle strutture condivise (reconfigtest).** Questo scenario verifica la stabilità del modulo quando le strutture dati condivise vengono accedute in maniera altamente concorrente. Mentre numerosi thread leggono ininterrottamente i filtri attivi (durante la fase di rate limiting), un thread parallelo aggiunge e rimuove costantemente UID e nomi di processo. L'obiettivo del test è mostrare che l'implementazione delle primitive di sincronizzazione è corretta e robusta: il completamento con successo certifica che questa intensa riconfigurazione non innesci *race condition*, corruzioni di memoria o *deadlock*, garantendo che il modulo non provochi un *Kernel Panic* durante il normale utilizzo.
- **Verifica del Rate-Limiting e reattività di spegnimento (monitortest).** Questo scenario verifica sia il ritmo di erogazione imposto dal *rate-limiter*, sia la reattività del sistema alla disattivazione. Il test imposta un limite volutamente basso (1 token al secondo), esaurisce subito la disponibilità iniziale e lancia molteplici thread che tentano di invocare la system call. Attraverso dei log a terminale, è possibile osservare i processi che notificano il proprio accodamento per poi venire sbloccati, uno alla volta, a intervalli regolari di un secondo; questo comportamento certifica la correttezza del *Token Bucket*. In una seconda fase, il test disattiva improvvisamente il monitoraggio: l'effetto generato è un risveglio istantaneo e simultaneo di tutti i thread residui. Misurando la latenza di questa operazione, si dimostra che il comando di spegnimento forza lo svuotamento immediato della coda, restituendo il controllo ai processi bloccati senza attendere i successivi scatti del timer.
- **Validazione dell'Authorization Model (permtest).** Verifica il corretto isolamento dei privilegi. Eseguito inizialmente con permessi di amministratore, il processo di test riduce volontariamente i propri privilegi richiamando `seteuid()` per simulare un utente standard. In questo stato declassato, il test accerta che ogni tentativo di alterare la configurazione tramite `ioctl` venga respinto dal kernel con codice `-EPERM`. Al contempo, conferma che le operazioni di sola lettura restino regolarmente accessibili, nel rispetto delle specifiche di progetto.
- **Robustezza e Boundary Testing (boundtest).** Sottopone l'interfaccia `ioctl` a una serie di input malformati, fuori scala o logicamente incoerenti. L'obiettivo è validare la solidità dei meccanismi di sanificazione in kernel-space: il test si assicura che il modulo intercetti correttamente le anomalie, respingendo i comandi con codici

di errore appropriati.

- **Protezione dello smontaggio (`unloadtest`).** Valuta la solidità del meccanismo di *reference counting* simulando tentativi forzati di rimozione del modulo in presenza di thread attivi. L'esperimento si sviluppa su due scenari distinti:
 1. *Thread accodati nel rate-limiter:* Verifica che il tentativo di scaricamento venga respinto con errore `-EBUSY` finché i processi rimangono sospesi nella *wait queue* interna del modulo.
 2. *Thread in esecuzione bloccante nativa:* Dimostra la copertura dell'hooking iniettando un'attesa prolungata tramite una chiamata nativa bloccante (`nanosleep`). Il test accerta che il kernel continui a respingere `rmmod` anche quando i thread hanno superato il motore di regolazione e si trovano addormentati all'interno della routine originale del kernel.

Una volta completate le chiamate e con il contatore dei riferimenti rientrato a zero, la procedura accerta che lo scaricamento finale avvenga regolarmente e che il device node `/dev/throttle_dev` sia rimosso senza generare instabilità o *Kernel Panic*.